Google Skills

The solution: Three components working together

How the architecture works

Now that you understand the problem of building without structure, let's see how the three-component architecture solves it. Here's how these components connect:

```
graph LR Input[User goal] -->|Enters agent| O

subgraph Agent["AI agent"]
    O["Orchestration<br/>(process)"]
    M["Model<br/>(thinking)"]
    T["Tools<br/>(doing)"]

    O <--> M
    O <--> T
    M -.-> T
end

T -->|Exits agent| Output[Goal achieved]

style M fill:#e1f5fe
style O fill:#f3e5f5
style T fill:#fff3e0
```

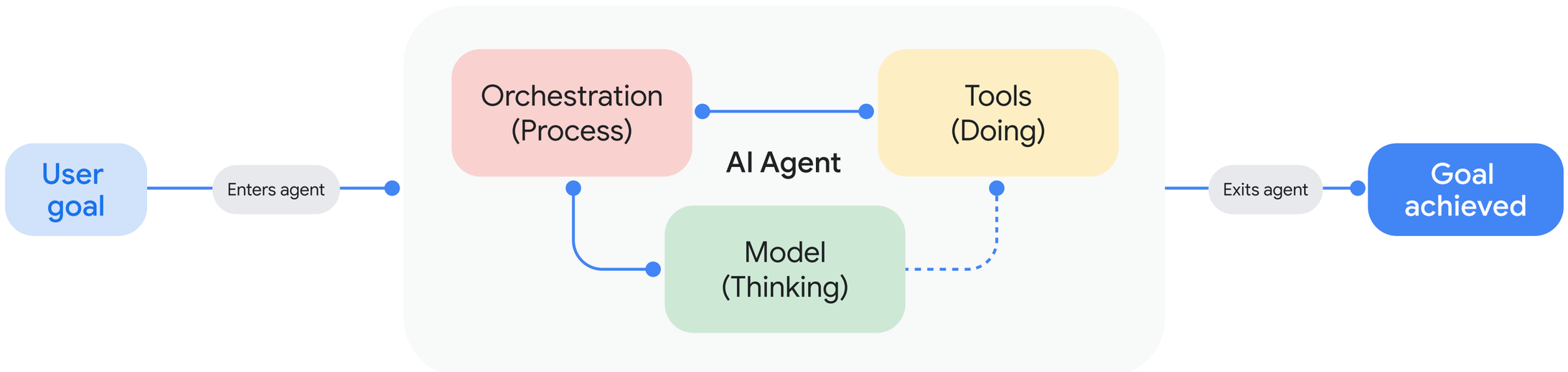


Figure: The three components working together in a continuous loop.

Notice how orchestration sits at the center, connecting the model's intelligence with the tools' real-world capabilities.

Let's examine each component in detail:

Component 1: The model (the brain)

According to the whitepaper, the model serves as the centralized decision maker for all agent processes. It's the reasoning engine that enables the agent to understand goals, make decisions, and adapt based on feedback.

The model does three critical things:

Understanding intent: Takes natural language and extracts the actual goal while handling ambiguity. When you say, "I need to be in Paris next week for the conference," the model structures this as: travel needed, destination unclear (Paris, France or Paris, Texas?), timeframe next week, and purpose conference. It recognizes the ambiguity and asks: "Which Paris do you mean—Paris, France or Paris, Texas?"

Making decisions: At each step, it decides what to do next. Should it search for flights first or check hotel availability? Should it ask for more information or proceed with defaults? These decisions require reasoning about priorities, dependencies, and likely outcomes.

Learning from feedback: When a tool returns results or an error occurs, the model interprets this feedback and adjusts its approach. Is the flight too expensive? Try different dates. API error? Try alternative service. This adaptive behavior is what makes agents robust.

Component 2: The tools (the hands)

If the model is the brain, tools are the hands; they let the agent actually do things in the world. Without tools, even the smartest model is just generating text about what it would do.

Here are three different examples of commonly used tools:

Pre-built integrations handle common tasks by connecting to external services. Someone has already figured out how to interact with systems like Google Flights or weather APIs, and packaged that knowledge so any agent can use it. The agent expresses what it needs—"search flights to Paris"—and the integration handles all the details of API calls and data formatting behind the scenes.

Custom functions allow the agent to leverage your existing application code. The agent determines that a specific operation should be performed and prepares the request, but your application actually executes it. This is perfect when you need control—like requiring human approval before making a purchase, or when you want to tap into business logic you've already written.

Information retrieval from databases lets agents access knowledge beyond their training data: your company's documentation, customer histories, product catalogs, and so on. Instead of the agent needing to know everything upfront, it knows how to find the right information when needed.

The true power lies in the agent's ability to orchestrate and chain multiple tool calls together to achieve a higher-level goal. An agent might retrieve information from a database to understand your preferences, use a pre-built integration to find flights matching those preferences, and prepare a custom function call for you to approve the purchase.

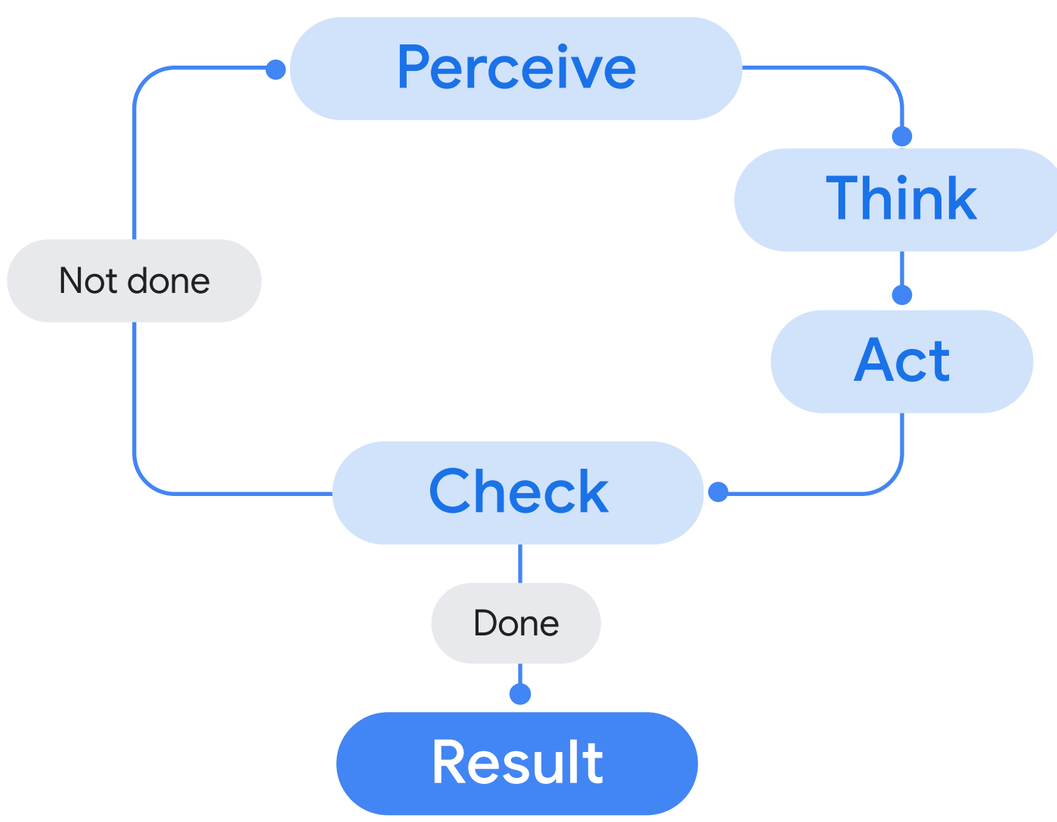
Component 3: The orchestration (the process)

In the previous section, you saw how an agent's true power lies in its ability to orchestrate and chain multiple tool calls together. But what exactly is this orchestration?

According to the whitepaper, the orchestration layer is a cyclical process that governs how the agent takes in information, performs internal reasoning, and uses that reasoning to inform its next action or decision. This loop continues until the agent reaches its goal.

The orchestration layer implements what we call the agent loop:

```
graph LR
    A[Perceive] --> B[Think]
    B --> C[Act]
    C --> D[Check]
    D -->|Not done| A
    D -->|Done| E[Result]
```



This loop continues until the agent achieves its goal. Let's see how it works with our "book a trip to Paris" example:

Perceive: The agent receives the request and understands the current situation—no flights booked, no hotels reserved, travel dates unclear.

Think: The agent reasons about what to do next. It needs dates first, so it should check the user's calendar for availability.

Act: The agent uses the calendar tool to find free periods next week.

Check: Are we done? No—we have dates but no bookings yet.

The loop continues: perceive the calendar results, think about finding flights for those dates, act by searching for flights, check if we're done. Round and round until the trip is fully booked.

The model can follow different reasoning frameworks to guide how it thinks and acts—techniques like ReAct (reasoning + acting), chain-of-thought, or tree-of-thoughts. These frameworks help the model decide what to do at each step of the loop, but the core pattern remains the same: perceive → think → act → check.

How the three components unite

Here's the beautiful part—these three components create a complete system:

The **model** receives a goal and breaks it down into steps. The **orchestration** manages the cycle of thinking and acting. The **tools** execute actions in the real world.

Together, they create an agent that can pursue goals autonomously. The model provides intelligence, tools provide capability, and orchestration provides the process that connects them.

Let's see this in action with our flight booking example:

```
User: "Book me a flight to Paris next week"

Loop Iteration 1:
├─ Perceive: User wants to book a flight to Paris next week
├─ Think: I need to know which Paris (France or Texas?) and specific dates
├─ Act: Ask clarifying question
└─ Check: Not done - need user response

User: "Paris, France. I'm flexible on dates."

Loop Iteration 2:
├─ Perceive: Destination is Paris, France; user has date flexibility
├─ Think: I should check the user's calendar for availability next week
├─ Act: Use calendar tool → Returns "Available Mon-Wed next week"
└─ Check: Not done - have dates but no flight options yet

Loop Iteration 3:
├─ Perceive: User available Mon-Wed next week
├─ Think: Search for flights on those dates
├─ Act: Use flight search tool → Returns 3 flight options
└─ Check: Not done - need user to choose

Agent: "I found 3 flights to Paris, France for Monday-Wednesday next week.
       Option 1: $450, direct flight...
       Option 2: $380, one stop...
       Option 3: $520, premium economy..."

User: "Book option 1"

Loop Iteration 4:
├─ Perceive: User selected option 1 ($450 direct flight)
├─ Think: Ready to book the selected flight
├─ Act: Use booking tool → Booking confirmed
└─ Check: Goal achieved - flight booked
Agent: "Done! Your direct flight to Paris, France is booked for Monday."
```

Notice how the orchestration manages the loop, the model makes decisions at each step (handling ambiguity, checking calendar, searching flights, and confirming booking), and the tools execute the actions. Each component plays its role, and together they produce the autonomous, goal-directed behavior that defines an agent.

Key takeaways

Understanding how agents work comes down to three core ideas:

- **Three components work together:** The model serves as the centralized decision maker, tools bridge the gap to the external world, and orchestration provides the cyclical process that connects them.
- **The agent loop drives behavior:** Perceive → think → act → check repeats across multiple iterations until the goal is achieved.
- **Tool chaining creates power:** Agents can orchestrate and chain multiple tool calls together, handling ambiguity and adapting based on feedback at each step.

The beauty of this architecture is how simple components working together create complex, autonomous behavior. The model provides intelligence, tools provide capability, and orchestration provides the process—together, they enable agents to pursue goals independently.